Mongo DB Tables and Schemas :

Tables:

announcements
calls
callparticipants
faqs
friends
groups
messages
notifications
reports
settings
statuses
stickers
ticketchats
tickets
user_deletions
users
wallpapers

Schema

announcements:

```
const AnnouncementSchema = new mongoose.Schema(
 {
   title: {
     type: String,
     required: [true, 'Title is required'],
     trim: true
   },
   content: {
     type: String,
     required: [true, 'Content is required']
   },
   media: {
     type: String,
     default: null
   },
   mediaType: {
```

```
      type: String,
      enum: ['image', 'video', 'pdf', null],
      default: null
    },
    status: {
      type: String,
      enum: ['Published', 'Draft', 'Inactive'],
      default: 'Draft'
    }
  },
  {
    timestamps: true
  }
);
```

call :

```
const CallSchema = new mongoose.Schema(
  {
    initiator_id: {
      type: mongoose.Schema.Types.ObjectId,
      ref: 'User',
      required: true
    },
    receiver_id: {
      type: mongoose.Schema.Types.ObjectId,
      ref: 'User',
      required: true
    },
    group_name: {
      type: String
    },
    is_group: {
      type: Boolean,
      default: false
    },
    group_id: {
      type: mongoose.Schema.Types.ObjectId,
      ref: 'Group'
    },
    group_avatar: {
      type: String
    },
    call_type: {
```

```
      type: String,
      enum: ['audio', 'video'],
      default: 'audio'
    },
    status: {
      type: String,
      enum: ['ringing', 'active', 'ended', 'missed', 'declined'],
      default: 'ringing'
    },
    started_at: {
      type: Date
    },
    ended_at: {
      type: Date
    },
    duration: {
      type: Number, // in seconds
      default: 0
    }
  },
  {
    timestamps: { createdAt: 'created_at', updatedAt: 'updated_at' }
  }
);
```

callparticipants

```
const CallParticipantSchema = new mongoose.Schema(
  {
    call_id: {
      type: mongoose.Schema.Types.ObjectId,
      ref: 'Call',
      required: true
    },
    user_id: {
      type: mongoose.Schema.Types.ObjectId,
      ref: 'User',
      required: true
    },
    status: {
      type: String,
      enum: ['invited', 'joined', 'declined', 'left'],
      default: 'invited'
```

```
    },
    joined_at: {
      type: Date
    },
    left_at: {
      type: Date
    },
    is_muted: {
      type: Boolean,
      default: false
    }
  },
  {
    timestamps: { createdAt: 'created_at', updatedAt: 'updated_at' }
  }
);
```

faqs:

```
const FAQSchema = new mongoose.Schema(
  {
    question: {
      type: String,
      required: [true, 'Question is required'],
      trim: true
    },
    answer: {
      type: String,
      required: [true, 'Answer is required']
    },
    status: {
      type: String,
      enum: ['Active', 'Inactive'],
      default: 'Active'
    }
  },
  {
    timestamps: { createdAt: 'created_at', updatedAt: 'updated_at' }
  }
);
```

friends:

```
const FriendSchema = new mongoose.Schema(
  {
```

```
    user_id: {
      type: mongoose.Schema.Types.ObjectId,
      ref: 'User',
      required: true
    },
    friend_id: {
      type: mongoose.Schema.Types.ObjectId,
      ref: 'User',
      required: true
    },
    status: {
      type: String,
      enum: ['pending', 'accepted', 'rejected', 'blocked'],
      default: 'pending'
    },
    requested_by: {
      type: mongoose.Schema.Types.ObjectId,
      ref: 'User',
      required: true
    },
    blocked_by: [{
      type: mongoose.Schema.Types.ObjectId,
      ref: 'User'
    }],
    blocked_users: [{
      type: mongoose.Schema.Types.ObjectId,
      ref: 'User'
    }],
    previous_status: {
      type: String,
      default: null
    }
  },
  {
    collection: 'friends',
    timestamps: { createdAt: 'created_at', updatedAt: 'updated_at' }
  }
);
```

groups :

```
const GroupSchema = new mongoose.Schema(
  {
    name: {
      type: String,
```

```
    required: [true, 'Group name is required'],
    trim: true
  },
  description: {
    type: String,
    default: ''
  },
  group_image: {
    type: String,
    default: null
  },
  admin_id: {
    type: mongoose.Schema.Types.ObjectId,
    ref: 'User',
    required: true
  },
  members: [
    {
      user_id: {
        type: mongoose.Schema.Types.ObjectId,
        ref: 'User'
      },
      role: {
        type: String,
        enum: ['admin', 'member'],
        default: 'member'
      },
      joined_at: {
        type: Date,
        default: Date.now
      },
      left_at: {
        type: Date,
        default: null
      }
    }
  ],
  status: {
    type: String,
    enum: ['Active', 'Archived'],
    default: 'Active'
  }
},
{
```

```
  collection: 'groups',
  timestamps: { createdAt: 'created_at', updatedAt: 'updated_at' }
 }
);
```

messages :

```
const MessageSchema = new mongoose.Schema(
 {
  sender_id: {
   type: mongoose.Schema.Types.ObjectId,
   ref: 'User',
   required: true
  },
  recipient_id: {
   type: mongoose.Schema.Types.ObjectId,
   ref: 'User',
   default: null
  },
  group_id: {
   type: mongoose.Schema.Types.ObjectId,
   ref: 'Group',
   default: null
  },
  content: {
   type: String,
   default: null
  },
  message_type: {
   type: String,
   enum: ['text', 'image', 'video', 'file', 'audio', 'sticker', 'link', 'system'],
   default: 'text'
  },
  file_url: {
   type: String,
   default: null
  },
  file_name: {
   type: String,
   default: null
  },
  moderation_status: {
   type: String,
   enum: ['Pending', 'Approved', 'Blocked', 'Escalate', 'Moderation'],
   default: 'Pending'
```

```
  },
  moderated_by: {
    type: mongoose.Schema.Types.Mixed,
    default: null
  },
  deleted_for: [
    {
      type: mongoose.Schema.Types.ObjectId,
      ref: 'User'
    }
  ],
  is_deleted_for_everyone: {
    type: Boolean,
    default: false
  },
  is_forwarded: {
    type: Boolean,
    default: false
  },
  parent_message_id: {
    type: mongoose.Schema.Types.ObjectId,
    ref: 'Message',
    default: null
  },
  is_edited: {
    type: Boolean,
    default: false
  },
  status: {
    type: String,
    enum: ['sent', 'delivered', 'read'],
    default: 'sent'
  },
  read_at: {
    type: Date,
    default: null
  },
  read_by: [
    {
      user_id: { type: mongoose.Schema.Types.ObjectId, ref: 'User' },
      read_at: { type: Date, default: Date.now }
    }
  ]
},
```

```
  {
    collection: 'messages',
    timestamps: { createdAt: 'created_at', updatedAt: 'updated_at' },
    toJSON: { virtuals: true },
    toObject: { virtuals: true }
  }
);
```

notifications

```
const notificationSchema = new mongoose.Schema({
  recipientId: {
    type: mongoose.Schema.Types.ObjectId,
    required: true
    // Ref removed to allow both User and AdminUser IDs
  },
  type: {
    type: String,
    enum: ['new_ticket', 'ticket_assigned', 'ticket_update', 'ticket_reply'],
    required: true
  },
  title: {
    type: String,
    required: true
  },
  message: {
    type: String,
    required: true
  },
  metadata: {
    ticketId: {
      type: mongoose.Schema.Types.ObjectId,
      ref: 'Ticket'
    },
    customTicketId: String // e.g. TKT-1234
  },
  isRead: {
    type: Boolean,
    default: false
  }
}, {
  timestamps: true
});
```

reports

```
const ReportSchema = new mongoose.Schema(
  {
    reporterId: {
      type: mongoose.Schema.Types.ObjectId,
      ref: 'User',
      required: true
    },
    reportedUserId: {
      type: mongoose.Schema.Types.ObjectId,
      ref: 'User',
      required: true
    },
    reason: {
      type: String,
      enum: ['Spam', 'Harassment', 'Inappropriate Content', 'Other'],
      required: [true, 'Reason is required']
    },
    description: {
      type: String,
      trim: true,
      maxlength: [500, 'Description cannot exceed 500 characters']
    },
    status: {
      type: String,
      enum: ['Pending', 'Resolved', 'Dismissed'],
      default: 'Pending'
    },
    adminNotes: {
      type: String,
      trim: true
    }
  },
  {
    timestamps: true
  }
);
```

settings

```
const SettingSchema = new mongoose.Schema(
  {
    // 1. Media & Document Control
```

```
docLimit: {
  type: String,
  default: '20MB'
},
audioLimit: {
  type: String,
  default: '15MB'
},
videoLimit: {
  type: String,
  default: '100MB'
},
imageLimit: {
  type: String,
  default: '10MB'
},
shareLimit: {
  type: Number,
  default: 10
},
msgLength: {
  type: Number,
  default: 4096
},
uploadTypes: {
  type: String,
  default: '*'
},

// 2. Call & Session Control
callTimeout: {
  type: Number,
  default: 60
},
statusExpiry: {
  type: Number,
  default: 24
},
sessionExpiry: {
  type: Number,
  default: 30
},

// 3. Group Management
```

```
    maxGroup: {
      type: Number,
      default: 50
    },
    maxGroupMembers: {
      type: Number,
      default: 256
    }
  },
  {
    timestamps: true
  }
);
```

statuses

```
const StatusSchema = new mongoose.Schema(
  {
    user_id: {
      type: mongoose.Schema.Types.ObjectId,
      ref: 'User',
      required: true
    },
    content_type: {
      type: String,
      enum: ['text', 'image', 'video'],
      required: true
    },
    content: {
      type: String,
      required: true
    },
    background_color: {
      type: String,
      default: '#000000'
    },
    caption: {
      type: String,
      default: ''
    },
    viewers: [
      {
        type: mongoose.Schema.Types.ObjectId,
        ref: 'User'
```

```
    }
   ],
   // The status will automatically be deleted after 24 hours (86400 seconds)
   expireAt: {
    type: Date,
    index: { expires: 0 }
   },
   created_at: {
    type: Date,
    default: Date.now
   },
   privacy_type: {
    type: String,
    enum: ['contacts', 'selective'],
    default: 'contacts'
   },
   privacy_list: [{
    type: mongoose.Schema.Types.ObjectId,
    ref: 'User'
   }]
  },
  {
   collection: 'statuses',
   timestamps: { createdAt: false, updatedAt: 'updated_at' }
  }
);
```

stickers

```
const StickerSchema = new mongoose.Schema(
  {
   name: {
    type: String,
    required: [true, 'Sticker name is required'],
    trim: true
   },
   sticker_image: {
    type: String,
    required: [true, 'Sticker image URL is required']
   },
   status: {
    type: String,
    enum: ['Active', 'Inactive'],
    default: 'Active'
```

```
    }
  },
  {
    timestamps: { createdAt: 'created_at', updatedAt: 'updated_at' }
  }
);
```

tickets

```
const TicketSchema = new mongoose.Schema(
  {
    ticketId: {
      type: String,
      unique: true,
      required: true
    },
    userId: {
      type: mongoose.Schema.Types.ObjectId,
      ref: 'User',
      required: true
    },
    subject: {
      type: String,
      required: [true, 'Subject is required'],
      trim: true
    },
    category: {
      type: String, // free text like "billing", "technical"
      required: true,
      trim: true
    },
    priority: {
      type: String,
      enum: ['High', 'Medium', 'Low']
      // default: 'Medium'
    },
    status: {
      type: String,
      enum: ['Open', 'Pending', 'Resolved', 'Closed'],
      default: 'Pending'
    },
    assignedTo: {
      type: mongoose.Schema.Types.ObjectId,
      ref: 'AdminUser',
      default: null
```

```
    }
  },
  {
    timestamps: true
  }
);
```

ticketchats

```
const TicketChatSchema = new mongoose.Schema(
  {
    ticketId: {
      type: mongoose.Schema.Types.ObjectId,
      ref: 'Ticket',
      required: true
    },
    adminUserId: {
      type: mongoose.Schema.Types.ObjectId,
      ref: 'AdminUser',
      default: null
    },
    userId: {
      type: mongoose.Schema.Types.ObjectId,
      ref: 'User',
      default: null
    },
    message: {
      type: String,
      trim: true
    },
    fileUrl: { type: String, default: null },
    fileName: { type: String, default: null },
    fileSize: { type: Number, default: null }
  },
  {
    timestamps: true
  }
);
```

users

```
const UserSchema = new mongoose.Schema(
  {
    name: {
      type: String,
      required: [true, 'Name is required'],
```

```
  trim: true
},
email: {
  type: String,
  required: [true, 'Email is required'],
  unique: true,
  trim: true,
  lowercase: true,
  match: [/^\w+([.-]?\w+)*@\w+([.-]?\w+)*(\.\w{2,3})+$/, 'Please fill a valid email address']
},
password: {
  type: String,
  required: [true, 'Password is required'],
  minlength: [6, 'Password must be at least 6 characters long'],
  select: false // Don't return password by default in queries
},
role: {
  type: String,
  enum: ['user', 'admin', 'Support Team'],
  default: 'user'
},
approval_status: {
  type: String,
  enum: ['Pending', 'Active', 'Inactive', 'Rejected', 'Suspended'],
  default: 'Pending'
},
avatar: {
  type: String,
  default: null
},
phone: {
  type: String,
  default: null
},
bio: {
  type: String,
  default: 'Available'
},
is_verified: {
  type: Boolean,
  default: false
},
last_login: {
  type: Date,
```

```
  default: null
},
is_online: {
  type: Boolean,
  default: false
},
last_seen: {
  type: Date,
  default: null
},
moderator_id: {
  type: mongoose.Schema.Types.ObjectId,
  ref: 'AdminUser',
  default: null
},
status_privacy: {
  type: String,
  enum: ['contacts', 'selective'],
  default: 'contacts'
},
status_privacy_list: [{
  type: mongoose.Schema.Types.ObjectId,
  ref: 'User'
}],
archived_chats: [{
  type: mongoose.Schema.Types.ObjectId
}],
favorite_chats: [{
  type: mongoose.Schema.Types.ObjectId
}],
show_last_seen: {
  type: Boolean,
  default: true
},
show_profile_pic: {
  type: Boolean,
  default: true
},
show_bio: {
  type: Boolean,
  default: true
},
show_read_receipts: {
  type: Boolean,
```

```
      default: true
    },
    show_typing_indicator: {
      type: Boolean,
      default: true
    },
    chat_wallpaper: {
      type: String,
      default: null
    },
    status_reason: {
      type: String,
      default: null
    },
    resetPasswordOTP: {
      type: String,
      default: null
    },
    resetPasswordExpires: {
      type: Date,
      default: null
    },
    fcmToken: {
      type: String,
      default: null
    }
  },
  {
    timestamps: true,
    toJSON: { virtuals: true },
    toObject: { virtuals: true }
  }
);
```

wallpapers

```
const WallpaperSchema = new mongoose.Schema(
  {
    name: {
      type: String,
      required: [true, 'Wallpaper name is required'],
      trim: true
    },
    wallpaper_image: {
```

```
      type: String,
      required: [true, 'Wallpaper image URL is required']
    },
    status: {
      type: String,
      enum: ['Active', 'Inactive'],
      default: 'Active'
    },
    isDefault: {
      type: Boolean,
      default: false
    }
  },
  {
    timestamps: { createdAt: 'created_at', updatedAt: 'updated_at' }
  }
);
```

user_deletions

```
const UserDeleteSchema = new mongoose.Schema(
  {
    user_id: {
      type: mongoose.Schema.Types.ObjectId,
      ref: 'User',
      required: true
    },
    chat_id: {
      type: mongoose.Schema.Types.ObjectId,
      required: true
    },
    chat_type: {
      type: String,
      enum: ['user', 'group'],
      required: true
    },
    deleted_at: {
      type: Date,
      default: Date.now
    }
  },
  {
    collection: 'user_deletions',
    timestamps: { createdAt: 'created_at', updatedAt: 'updated_at' }
  }
```

);